# 2

# Basic principles

*The aim of this chapter is to lay the groundwork for the more detailed information presented in the following chapters.*

## Objectives

When you have completed study of this chapter you will be able to:

- Explain the basics of the binary numbering system – bits, bytes and characters
- Describe the factors that affect transmission speed:
  - Bandwidth
  - Signal-to-noise ratio
  - Data throughput
  - Error rate
- Explain the basic components of a communication system
- Describe the three communication modes
- Describe the message format and error detection in asynchronous communication systems
- List and explain the most common data codes:
  - Baudot
  - ASCII
  - EBCDIC
  - 4-bit binary code
  - Gray code
  - Binary coded decimal (BCD)
- Describe the message format and error detection in synchronous communication systems
- Describe the universal asynchronous transmitter/receiver

## 2.1 Bits, bytes and characters

A computer uses the binary numbering system, which has only two digits, 0 and 1. Any number can be represented by a string of these digits, known as bits (from binary digit). For example, the decimal number 5 is equal to the binary number 101.

| | |
|---|---|
| Bit | 1 or a 0 |
| Dibit | two bits (10) |
| Nibble | four bits (1001 or one Hex character) |
| Byte | eight bits or two nibbles (11000001, C1 Hex) |
| Word | the width of the bus of the computer |

**Table 2.1**
*Different sets of bits*

As a bit can have only two values, it can be represented by a voltage that is either on (1) or off (0). This is also known as logical 1 and logical 0. Typical values used in a computer are 0 V for logical 0 and +5 V for logical 1, although it could also be the other way around i.e. 0 V for 1 and +5 V for 0.

A string of eight bits is called a 'byte' (or octet), and can have values ranging from 0 (0000 0000) to $255_{10}$ ($1111\ 1111_2$). Computers generally manipulate data in bytes or multiples of bytes.

| Decimal | Hexadecimal | Binary |
|---|---|---|
| 0 | 0 | 0000 |
| 1 | 1 | 0001 |
| 2 | 2 | 0010 |
| 3 | 3 | 0011 |
| 4 | 4 | 0100 |
| 5 | 5 | 0101 |
| 6 | 6 | 0110 |
| 7 | 7 | 0111 |
| 8 | 8 | 1000 |
| 9 | 9 | 1001 |
| 10 | A | 1010 |
| 11 | B | 1011 |
| 12 | C | 1100 |
| 13 | D | 1101 |
| 14 | E | 1110 |
| 15 | F | 1111 |

**Table 2.2**
*The hexadecimal table*

Programmers use ‘hexadecimal’ notation because it is a more convenient way of defining and dealing with bytes. In the hexadecimal numbering system, there are 16 digits (0–9 and A–F) each of which is represented by four bits. A byte is therefore represented by two hexadecimal digits.

A ‘character’ is a symbol that can be printed. The alphabet, both upper and lower case, numerals, punctuation marks and symbols such as ‘*’ and ‘&’ are all characters. A computer needs to express these characters in such a way that they can be understood by other computers and devices. The most common code for achieving this is the American Standard Code for Information Interchange (ASCII) described in section 2.8.

## 2.2 Communication principles

Every data communications system requires:

- A source of data (a transmitter or line driver), which converts the information into a form suitable for transmission over a link
- A receiver that accepts the signal and converts it back into the original data
- A communications link that transports the signals. This can be copper wire, optical fiber, and radio or satellite link

In addition, the transmitter and receiver must be able to understand each other. This requires agreement on a number of factors. The most important are:

- The type of signaling used
- Defining a logical ‘1’ and a logical ‘0’
- The codes that represent the symbols
- Maintaining synchronization between transmitter and receiver
- How the flow of data is controlled, so that the receiver is not swamped
- How to detect and correct transmission errors

The physical factors are referred to as the ‘interface standard’; the other factors comprise the ‘protocols’.

The physical method of transferring data across a communication link varies according to the medium used. The binary values 0 and 1, for example, can be signaled by the presence or absence of a voltage on a copper wire, by a pair of audio tones generated and decoded by a modem in the case of the telephone system, or by the use of modulated light in the case of optical fiber.

## 2.3 Communication modes

In any communications link connecting two devices, data can be sent in one of three communication modes. These are:

- Simplex
- Half duplex
- Full duplex

A simplex system is one that is designed for sending messages in one direction only. This is illustrated in Figure 2.1.

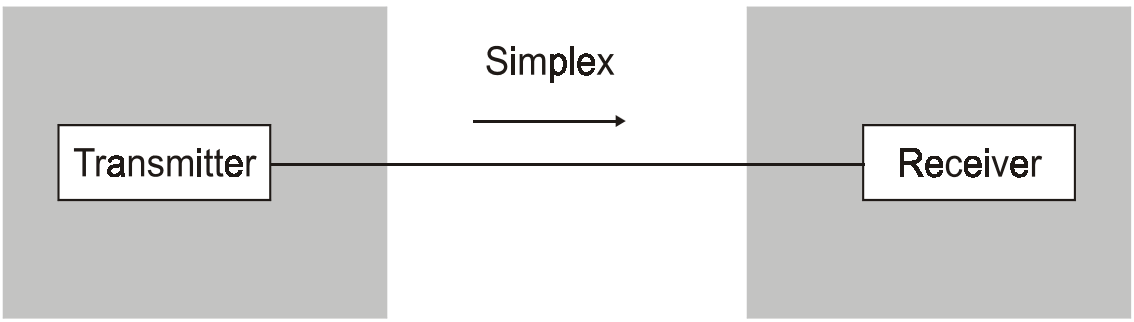



**Figure 2.1**
*Simplex communications*

A duplex system is designed for sending messages in both directions.

Half duplex occurs when data can flow in both directions, but in only one direction at a time (Figure 2.2).

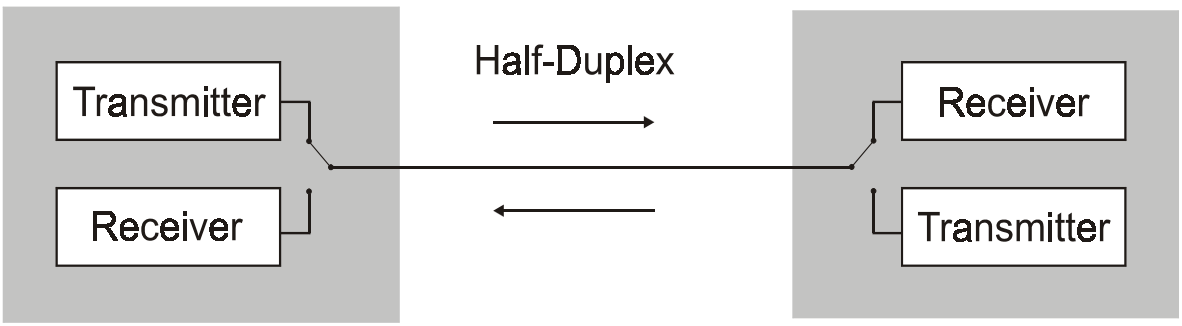



**Figure 2.2**
*Half-duplex communications*

In a full-duplex system, the data can flow in both directions simultaneously (Figure 2.3).

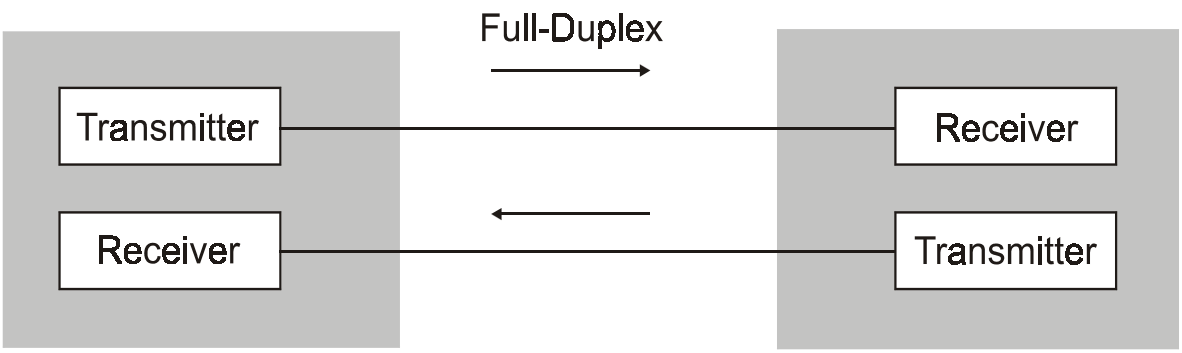



**Figure 2.3**
*Full duplex communications*

## 2.4 Asynchronous systems

An asynchronous system is one in which each character or byte is sent within a frame. The receiver does not start detection until it receives the first bit, known as the ‘start bit’.

The start bit is in the opposite voltage state to the idle voltage and allows the receiver to synchronize to the transmitter for the following data in the frame.

The receiver reads in the individual bits of the frame as they arrive, seeing either the logic 0 voltage or the logic 1 voltage at the appropriate time. The 'clock' rate at each end must be the same so that the receiver looks for each bit at the time the transmitter sends it. However, as the clocks are synchronized at the start of each frame, some variation can be tolerated at lower transmission speeds. The allowable variation decreases as data transmission rates increase, and asynchronous communication can have problems at high speeds (above 100 kbps).

### Message format

An asynchronous frame may have the following format:

| | |
|---|---|
| Start bit: | Signals the start of the frame |
| Data: | Usually 7 or 8 bits of data, but can be 5 or 6 bits |
| Parity bit: | Optional error detection bit |
| Stop bit(s): | Usually 1, 1.5 or 2 bits. A value of 1.5 means that the level is held for 1.5 times as long as for a single bit |

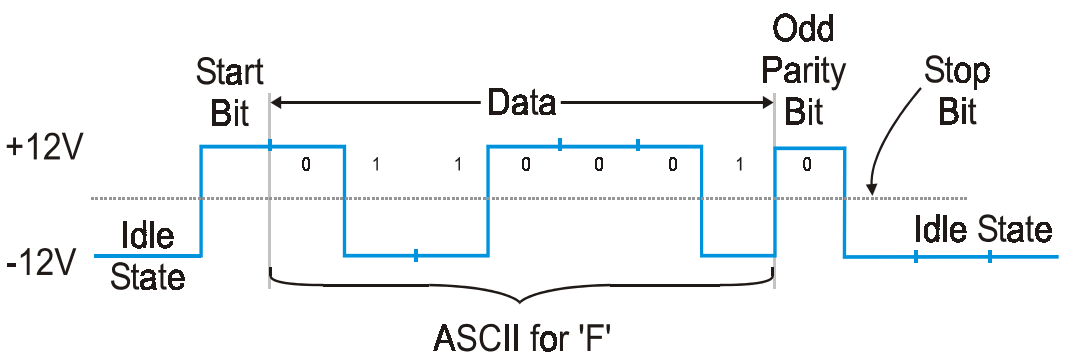



**Figure 2.4**
*Asynchronous frame format*

An asynchronous frame format is shown in Figure 2.4. The transmitter and receiver must be set to exactly the same configuration so that the data can be correctly extracted from the frame. As each character has its own frame, the actual data transmission speed is less than the bit rate. For example, with a start bit, seven data bits, one parity bit and one stop bit, there are ten bits needed to send seven bits of data. Thus the transmission of useful data is 70% of the overall bit rate.

## 2.5 Synchronous systems

In synchronous systems, the receiver initially synchronizes to the transmitter's clock pulses, which are incorporated in the transmitted data stream. This enables the receiver to maintain its synchronization throughout large messages, which could typically be up to 4500 bytes (36 000 bits). This allows large frames to be transmitted efficiently at high data rates. The synchronous system packs many characters together and sends them as a continuous stream, called a packet or a frame.

### Message format

A typical synchronous system frame format is shown below in Figure 2.5.

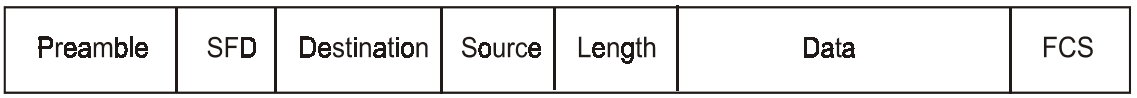



**Figure 2.5**
*Typical synchronous system frame format*

| | |
|---|---|
| Preamble: | This comprises one or more bytes that allow the receiving unit to synchronize with the frame. |
| SFD: | The start of frame delimiter signals the beginning of the frame. |
| Destination: | The address to which the frame is sent. |
| Source: | The address from which the frame originated. |
| Length: | The number of bytes in the data field. |
| Data: | The actual message. |
| FCS: | The frame check sequence is for error detection. |

Each of these is called a field.

## 2.6 Error detection

All practical data communications channels are subject to noise, particularly copper cables in industrial environments with high electrical noise. Refer to Chapter 6 for a separate discussion on noise. Noise can result in incorrect reception of the data.

The basic principle of error detection is for the transmitter to compute a check character based on the original message content. This is sent to the receiver on the end of the message and the receiver repeats the same calculation on the bits it receives. If the computed check character does not match the one sent, we assume an error has occurred. The various methods of error detection are covered in Chapter 4.

The simplest form of error checking in asynchronous systems is to incorporate a parity bit, which may be even or odd.

Even parity requires the total number of data bits at logic 1 plus the parity bit to equal an even number. The communications hardware at the transmission end calculates the parity required and sets the parity bit to give an even number of logic 1 bits.

Odd parity works in the same way as even parity, except that the parity bit is adjusted so that the total number of logic 1 bits, including the parity bit, equals an odd number.

The hardware at the receiving end determines the total number of logic 1 bits and reports an error if it is not an appropriate even or odd number. The receiver hardware also detects receiver overruns and frame errors.

Statistically, use of a parity bit has only about a 50% chance of detecting an error on a high speed system. This method can detect an odd number of bits in error and will not detect an even number of bits in error. The parity bit is normally omitted if there are more sophisticated error checking schemes in place.

## 2.7 Transmission characteristics

### Signaling rate (or baud rate)

The signaling rate of a communications link is a measure of how many times the physical signal changes per second and is expressed as the baud rate. An oscilloscope trace of the data transfer would show pulses at the baud rate. For a 1000 baud rate, pulses would be seen at multiples of 1 ms.

With asynchronous systems, we set the baud rate at both ends of the link so that each physical pulse has the same duration.

### Data rate

The data rate or bit rate is expressed in bits per second (bps), or multiples such as kbps, Mbps and Gbps (kilo, mega and gigabits per second). This represents the actual number of data bits transferred per second. An example is a 1000 baud RS-232 link transferring a frame of 10 bits, being 7 data bits plus a start, stop and parity bit. Here the baud rate is 1000 baud, but the data rate is 700 bps.

Although there is a tendency to confuse baud rate and bit rate, they are not the same. Whereas baud rate indicates the number of signal changes per second, the bit rate indicates the number of bits represented by each signal change. In simple baseband systems such as RS-232, the baud rate equals the bit rate. For synchronous systems, the bit rate invariably exceeds the baud rate. For ALL systems, the data rate is less than the bit rate due to overheads such as stop, stand, and parity bits (synchronous systems) or fields such as address and error detection fields in synchronous system frames.

There are sophisticated modulation techniques, used particularly in modems that allow more than one bit to be encoded within a signal change. The ITU V.22bis full duplex standard, for example, defines a technique called quadrature amplitude modulation, which effectively increases a baud rate of 600 to a data rate of 2400 bps. Irrespective of the methods used, the maximum data rate is always limited by the bandwidth of the link. These modulation techniques used with modems are discussed in Chapter 7.

### Bandwidth

The single most important factor that limits communication speeds is the bandwidth of the link. Bandwidth is generally expressed in hertz (Hz), meaning cycles per second. This represents the maximum frequency at which signal changes can be handled before attenuation degrades the message. Bandwidth is closely related to the transmission medium, ranging from around 5000 Hz for the public telephone system to the GHz range for optical fiber cable.

As a signal tends to attenuate over distance, communications links may require repeaters placed at intervals along the link, to boost the signal level.

Calculation of the theoretical maximum data transfer rate uses the Nyquist formula and involves the bandwidth and the number of levels encoded in each signaling element, as described in Chapter 4.

### Signal to noise ratio

The signal to noise (S/N) ratio of a communications link is another important limiting factor. Sources of noise may be external or internal, as discussed in Chapter 6.

The maximum practical data transfer rate for a link is mathematically related to the bandwidth, S/N ratio and the number of levels encoded in each signaling element. As the S/N decreases, so does the bit rate. See Chapter 4 for a definition of the Shannon-Hartley Law that gives the relationships.

### Data throughput

As data is always carried within a protocol envelope, ranging from a character frame to sophisticated message schemes, the data transfer rate will be less than the bit rate. As explained in Chapter 9, the amount of redundant data around a message packet increases as it passes down the protocol stack in a network. This means that the ratio of non-message data to 'real' information may be a significant factor in determining the effective transmission rate, sometimes referred to as the throughput.

### Error rate

Error rate is related to factors such as S/N ratio, noise, and interference. There is generally a compromise between transmission speed and the allowable error rate, depending on the type of application. Ordinarily, an industrial control system cannot allow errors and is designed for maximum reliability of data transmission. This means that an industrial system will be comparatively slow in data transmission terms. As data transmission rates increase, there is a point at which the number of errors becomes excessive. Protocols handle this by requesting a retransmission of packets. Obviously, the number of retransmissions will eventually reach the point at which a high apparent data rate actually gives a lower real message rate, because much of the time is being used for retransmission.

## 2.8 Data coding

An agreed standard code allows a receiver to understand the messages sent by a transmitter. The number of bits in the code determines the maximum number of unique characters or symbols that can be represented. The most common codes are described on the following pages.

### Baudot code

Although not in use much today, the Baudot code is of historical importance. It was invented in 1874 by Maurice Emile Baudot and is considered to be the first uniform-length code. Having five bits, it can represent 32 ($2^5$) characters and is suitable for use in a system requiring only letters and a few punctuation and control codes. The main use of this code was in early teleprinter machines.

A modified version of the Baudot code was adopted by the ITU as the standard for telegraph communications. This uses two 'shift' characters for letters and numbers and was the forerunner for the modern ASCII and EBCDIC codes.

### ASCII code

The most common character set in the western world is the American Standard Code for Information Interchange, or ASCII (see Table 2.3).

This code uses a 7-bit string giving 128 ($2^7$) characters, consisting of:

- Upper and lower case letters

- Numerals 0 to 9
- Punctuation marks and symbols
- A set of control codes, consisting of the first 32 characters, which are used by the
- Communications link itself and are not printable

For example: D = ASCII code in binary 1000100.

A communications link setup for 7-bit data strings can only handle hexadecimal values from 00 to 7F. For full hexadecimal data transfer, an 8-bit link is needed, with each packet of data consisting of a byte (two hexadecimal digits) in the range 00 to FF. An 8-bit link is often referred to as 'transparent' because it can transmit any value. In such a link, a character can still be interpreted as an ASCII value if required, in which case the eighth bit is ignored.

The full hexadecimal range can be transmitted over a 7-bit link by representing each hexadecimal digit as its ASCII equivalent. Thus the hexadecimal number 8E would be represented as the two ASCII values 38 45 (hexadecimal) ('8' 'E'). The disadvantage of this technique is that the amount of data to be transferred is almost doubled, and extra processing is required at each end.

ASCII control codes can be accessed directly from a PC keyboard by pressing the Control key [Ctrl] together with another key. For example, Control-A (^A) generates the ASCII code start of header (SOH).

The ASCII Code is the most common code used for encoding characters for data communications. It is a 7-bit code and, consequently, there are only $2^7 = 128$ possible combinations of the seven binary digits (bits), ranging from binary 0000000 to 1111111 or hexadecimal 00 to 7F.

Each of these 128 codes is assigned to specific control codes or characters as specified by the following standards:

- ANSI-X3.4
- ISO-646
- ITU alphabet #5

The ASCII Table is the **reference** table used to record the bit value of every character defined by the code. There are many different forms of the table, but all contain the same basic information according to the standards. Two types are shown here.

Table 2.3 shows the condensed form of the ASCII Table, where all the characters and control codes are presented on one page. This table shows the code for each character in hexadecimal (HEX) and binary digits (BIN) values. Sometimes the decimal (DEC) values are also given in small numbers in each box.

This table works like a matrix, where the MSB (most significant bits – the digits on the left-hand side of the written HEX or BIN codes) are along the top of the table and the LSB (least significant bits – the digits on the right-hand side of the written HEX or BIN codes) are down the left-hand side of the table. Some examples of the HEX and BIN values are given below:

Table 2.4 and Table 2.5 show the form commonly used in printer manuals, sometimes also called the ASCII Code Conversion Table, where each ASCII character or control code is cross referenced to:

- BIN : A 7-bit binary ASCII code
- DEC : An equivalent 3 digit decimal value (0 to 127)
- HEX : An equivalent 2 digit hexadecimal value (00 to 7F)

| | HEX | 0 | 1 | 2 | 3 | 4 | 5 | 6 | **MSB** 7 |
|---|---|---|---|---|---|---|---|---|---|
| HEX | **BIN** | **000** | **001** | **010** | **011** | **100** | **101** | **110** | **111** |
| 0 | **0000** | (NUL) | (DLE) | Space | 0 | @ | P | ` | p |
| 1 | **0001** | (SOH) | (DC1) | ! | 1 | A | Q | a | q |
| 2 | **0010** | (STX) | (DC2) | " | 2 | B | R | b | r |
| 3 | **0011** | (ETX) | (DC3) | # | 3 | C | S | c | s |
| 4 | **0100** | (EOT) | (DC4) | $ | 4 | D | T | d | t |
| 5 | **0101** | (ENQ) | (NAK) | % | 5 | E | U | e | u |
| 6 | **0110** | (ACK) | (SYN) | & | 6 | F | V | f | v |
| 7 | **0111** | (BEL) | (ETB) | ' | 7 | G | W | g | w |
| 8 | **1000** | (BS) | (CAN) | ( | 8 | H | X | h | x |
| 9 | **1001** | (HT) | (EM) | ) | 9 | I | Y | i | y |
| A | **1010** | (LF) | (SUB) | * | : | J | Z | j | z |
| B | **1011** | (VT) | (ESC) | + | ; | K | [ | k | { |
| C | **1100** | (FF) | (FS) | , | < | L | \ | l | \| |
| D | **1101** | (CR) | (GS) | - | = | M | ] | m | } |
| E | **1110** | (SO) | (RS) | . | > | N | ^ | n | ~ |
| F **LSB** | **1111** | (SI) | (US) | / | ? | O | _ | o | DEL |

**Table 2.3**
*The ASCII table*

| DEC | HEX | BIN | ASCII |
|---|---|---|---|
| 0 | 00 | 000 0000 | (NUL) |
| 1 | 01 | 000 0001 | (SOH) |
| 2 | 02 | 000 0010 | (STX) |
| 3 | 03 | 000 0011 | (ETX) |
| 4 | 04 | 000 0100 | (EOT) |
| 5 | 05 | 000 0101 | (ENQ) |
| 6 | 06 | 000 0110 | (ACK) |
| 7 | 07 | 000 0111 | (BEL) |
| 8 | 08 | 000 1000 | (BS) |
| 9 | 09 | 000 1001 | (HT) |
| 10 | 0A | 000 1010 | (LF) |
| 11 | 0B | 000 1011 | (VT) |
| 12 | 0C | 000 1100 | (FF) |
| 13 | 0D | 000 1101 | (CR) |
| 14 | 0E | 000 1110 | (SO) |
| 15 | 0F | 000 1111 | (SI) |
| 16 | 10 | 001 0000 | (DLE) |
| 17 | 11 | 001 0001 | (DC1) |
| 18 | 12 | 001 0010 | (DC2) |
| 19 | 13 | 001 0011 | (DC3) |
| 20 | 14 | 001 0100 | (DC4) |
| 21 | 15 | 001 0101 | (NAK) |
| 22 | 16 | 001 0110 | (SYN) |
| 23 | 17 | 001 0111 | (ETB) |
| 24 | 18 | 001 1000 | (CAN) |
| 25 | 19 | 001 1001 | (EM) |
| 26 | 1A | 001 1010 | (SUB) |
| 27 | 1B | 001 1011 | (ESC) |
| 28 | 1C | 001 1100 | (FS) |
| 29 | 1D | 001 1101 | (GS) |
| 30 | 1E | 001 1110 | (RS) |
| 31 | 1F | 001 1111 | (US) |

| DEC | HEX | BIN | ASCII |
|---|---|---|---|
| 32 | 20 | 010 0000 | Space |
| 33 | 21 | 010 0001 | ! |
| 34 | 22 | 010 0010 | " |
| 35 | 23 | 010 0011 | # |
| 36 | 24 | 010 0100 | $ |
| 37 | 25 | 010 0101 | % |
| 38 | 26 | 010 0110 | & |
| 39 | 27 | 010 0111 | ' |
| 40 | 28 | 010 1000 | ( |
| 41 | 29 | 010 1001 | ) |
| 42 | 2A | 010 1010 | * |
| 43 | 2B | 010 1011 | + |
| 44 | 2C | 010 1100 | , |
| 45 | 2D | 010 1101 | - |
| 46 | 2E | 010 1110 | . |
| 47 | 2F | 010 1111 | / |
| 48 | 30 | 011 0000 | 0 |
| 49 | 31 | 011 0001 | 1 |
| 50 | 32 | 011 0010 | 2 |
| 51 | 33 | 011 0011 | 3 |
| 52 | 34 | 011 0100 | 4 |
| 53 | 35 | 011 0101 | 5 |
| 54 | 36 | 011 0110 | 6 |
| 55 | 37 | 011 0111 | 7 |
| 56 | 38 | 011 1000 | 8 |
| 57 | 39 | 011 1001 | 9 |
| 58 | 3A | 011 1010 | : |
| 59 | 3B | 011 1011 | ; |
| 60 | 3C | 011 1100 | < |
| 61 | 3D | 011 1101 | = |
| 62 | 3E | 011 1110 | > |
| 63 | 3F | 011 1111 | ? |

**Table 2.4**
*ASCII code conversion table*

| DEC | HEX | BIN | ASCII |
|---|---|---|---|
| 64 | 40 | 100 0000 | @ |
| 65 | 41 | 100 0001 | A |
| 66 | 42 | 100 0010 | B |
| 67 | 43 | 100 0011 | C |
| 68 | 44 | 100 0100 | D |
| 69 | 45 | 100 0101 | E |
| 70 | 46 | 100 0110 | F |
| 71 | 47 | 100 0111 | G |
| 72 | 48 | 100 1000 | H |
| 73 | 49 | 100 1001 | I |
| 74 | 4A | 100 1010 | J |
| 75 | 4B | 100 1011 | K |
| 76 | 4C | 100 1100 | L |
| 77 | 4D | 100 1101 | M |
| 78 | 4E | 100 1110 | N |
| 79 | 4F | 100 1111 | O |
| 80 | 50 | 101 0000 | P |
| 81 | 51 | 101 0001 | Q |
| 82 | 52 | 101 0010 | R |
| 83 | 53 | 101 0011 | S |
| 84 | 54 | 101 0100 | T |
| 85 | 55 | 101 0101 | U |
| 86 | 56 | 101 0110 | V |
| 87 | 57 | 101 0111 | W |
| 88 | 58 | 101 1000 | X |
| 89 | 59 | 101 1001 | Y |
| 90 | 5A | 101 1010 | Z |
| 91 | 5B | 101 1011 | [ |
| 92 | 5C | 101 1100 | \ |
| 93 | 5D | 101 1101 | ] |
| 94 | 5E | 101 1110 | ^ |
| 95 | 5F | 101 1111 | _ |

| DEC | HEX | BIN | ASCII |
|---|---|---|---|
| 96 | 60 | 110 0000 | ` |
| 97 | 61 | 110 0001 | a |
| 98 | 62 | 110 0010 | b |
| 99 | 63 | 110 0011 | c |
| 100 | 64 | 110 0100 | d |
| 101 | 65 | 110 0101 | e |
| 102 | 66 | 110 0110 | f |
| 103 | 67 | 110 0111 | g |
| 104 | 68 | 110 1000 | h |
| 105 | 69 | 110 1001 | i |
| 106 | 6A | 110 1010 | j |
| 107 | 6B | 110 1011 | k |
| 108 | 6C | 110 1100 | l |
| 109 | 6D | 110 1101 | m |
| 110 | 6E | 110 1110 | n |
| 111 | 6F | 110 1111 | o |
| 112 | 70 | 111 0000 | p |
| 113 | 71 | 111 0001 | q |
| 114 | 72 | 111 0010 | r |
| 115 | 73 | 111 0011 | s |
| 116 | 74 | 111 0100 | t |
| 117 | 75 | 111 0101 | u |
| 118 | 76 | 111 0110 | v |
| 119 | 77 | 111 0111 | w |
| 120 | 78 | 111 1000 | x |
| 121 | 79 | 111 1001 | y |
| 122 | 7A | 111 1010 | z |
| 123 | 7B | 111 1011 | { |
| 124 | 7C | 111 1100 | \| |
| 125 | 7D | 111 1101 | } |
| 126 | 7E | 111 1110 | ~ |
| 127 | 7F | 111 1111 | (DEL) |

**Table 2.5**
*ASCII code conversion table (cont.)*

| Character | | Control | 7-Bit Binary Code | Hex | Decimal |
|---|---|---|---|---|---|
| NUL | Null | ^@ | 000 0000 | 00 | 0 |
| SOH | Start of Header | ^A | 000 0001 | 01 | 1 |
| STX | Start of Text | ^B | 000 0010 | 02 | 2 |
| ETX | End of Text | ^C | 000 0011 | 03 | 3 |
| EOT | End of Transmission | ^D | 000 0100 | 04 | 4 |
| ENQ | Enquiry | ^E | 000 0101 | 05 | 5 |
| ACK | Acknowledge | ^F | 000 0110 | 06 | 6 |
| BEL | Bell | ^G | 000 0111 | 07 | 7 |
| BS | Backspace | ^H | 000 1000 | 08 | 8 |
| HT | Horizontal Tabulation | ^I | 000 1001 | 09 | 9 |
| LF | Line feed | ^J | 000 1010 | 0A | 10 |
| VT | Vertical Tabulation | ^K | 000 1011 | 0B | 11 |
| FF | Form Feed | ^L | 000 1100 | 0C | 12 |
| CR | Carriage return | ^M | 000 1101 | 0D | 13 |
| SO | Shift Out | ^N | 000 1110 | 0E | 14 |
| SI | Shift In | ^O | 000 1111 | 0F | 15 |
| DLE | Data Link Escape | ^P | 001 0000 | 10 | 16 |
| DC1 | Device Control 1 | ^Q | 001 0001 | 11 | 17 |
| DC2 | Device Control 2 | ^R | 001 0010 | 12 | 18 |
| DC3 | Device Control 3 | ^S | 001 0011 | 13 | 19 |
| DC4 | Device Control 4 | ^T | 001 0100 | 14 | 20 |
| NAK | Negative Acknowledgement | ^U | 001 0101 | 15 | 21 |
| SYN | Synchronous Idle | ^V | 001 0110 | 16 | 22 |
| ETB | End of Trans Block | ^W | 001 0111 | 17 | 23 |
| CAN | Cancel | ^X | 001 1000 | 18 | 234 |
| EM | End of Medium | ^Y | 001 1001 | 19 | 25 |
| SUB | Substitute | ^Z | 001 1010 | 1A | 26 |
| ESC | Escape | ^[ | 001 1011 | 1B | 27 |
| FS | File Separator | ^\ | 001 1100 | 1C | 28 |
| GS | Group Separator | ^] | 001 1101 | 1D | 29 |
| RS | Record Separator | ^\| | 001 1110 | 1E | 30 |
| US | Unit Separator | ^_ | 001 1111 | 1F | 31 |
| DEL | Delete, Rubout | | 111 1111 | 7F | 127 |

**Table 2.6**
*Table of control codes for the ASCII*

Least significant bits

| Bit positions | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| | | | 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| | | | 2 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| | | | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| 8 | 7 | 6 | 5 | | | | | | | | | | | | | | | | |
| 0 | 0 | 0 | 0 | NUL | SOH | STX | ETX | PF | HT | LC | DEL | | | SMM | VT | FF | CR | SO | SI |
| 0 | 0 | 0 | 1 | DLE | $DC_1$ | $DC_2$ | $DC_3$ | RES | NL | BS | IL | CAN | EM | CC | | IFS | IGS | IRS | IUS |
| 0 | 0 | 1 | 0 | DS | SOS | FS | | BYP | LF | EOB | PRE | | | SM | | | ENQ | ACK | BEL |
| 0 | 0 | 1 | 1 | | | SYN | | PN | RS | UC | EOT | | | | | DC4 | NAK | | SUB |
| 0 | 1 | 0 | 0 | SP | | | | | | | | | | ¢ | . | < | ( | + | \| |
| 0 | 1 | 0 | 1 | & | | | | | | | | | | ! | $ | * | ) | ; | _ |
| 0 | 1 | 1 | 0 | - | / | | | | | | | | | | ‘ | % | - | > | ? |
| 0 | 1 | 1 | 1 | | | | | | | | | | | : | # | @ | ‘ | = | “ |
| 1 | 0 | 0 | 0 | | a | b | c | d | e | f | g | h | i | | | | | | |
| 1 | 0 | 0 | 1 | | j | k | l | m | n | o | p | q | r | | | | | | |
| 1 | 0 | 1 | 0 | | | s | t | u | v | w | x | y | z | | | | | | |
| 1 | 0 | 1 | 1 | | | | | | | | | | | | | | | | |
| 1 | 1 | 0 | 0 | | A | B | C | D | E | F | G | H | I | | | | | | |
| 1 | 1 | 0 | 1 | | J | K | L | M | N | O | P | Q | R | | | | | | |
| 1 | 1 | 1 | 0 | | | S | T | U | V | W | X | Y | Z | | | | | | |
| 1 | 1 | 1 | 1 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | | | | | | |

Most significant bits

**Table 2.7**
*EBCDIC code table*

Control codes are often difficult to detect when troubleshooting a data system, unlike printable codes, which show up as a symbol on the printer or terminal. Digital line analyzers can be used to detect and display the unique code for each of these control codes to assist in the analysis of the system.

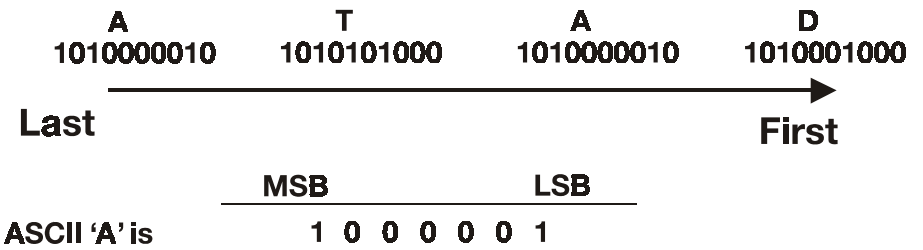
To represent the word DATA in binary form using the 7-bit ASCII code, each letter is coded as follows:

| | | ***Binary*** | ***Hex*** |
|---|---|---|---|
| D | : | 100 0100 | 44 |
| A | : | 100 0001 | 41 |
| T | : | 101 0100 | 54 |
| A | : | 100 0001 | 41 |

Referring to the ASCII table, the binary digits on the right-hand side of the binary column change by one digit for each step down the table. Consequently, the bit on the far right has become known as **least significant bit (LSB)** because it changes the overall value so little. The bit on the far left has become known as **most significant bit (MSB)** because it changes the overall value so much.

According to the reading conventions in the western world, words and sentences are read from left to right. When looking at the ASCII code for a character, we would read the MSB (most significant bit) first, which is on the left-hand side. However, in data communications, the convention is to **transmit the LSB of each character FIRST,** which is on the right-hand side and the **MSB last.** However, the **characters** are still usually sent in the conventional reading sequence in which they are generated. For example, if the word D-A-T-A is to be transmitted, the characters are transferred in that sequence, but the 7 bit ASCII code for each character is ‘reversed’.

Consequently, the bit pattern that is observed on the communication link will be as follows, reading each bit in order from right to left.




Adding the stop bit (1) and parity bit (1 or 0) and the start bit (0) to the ASCII character, the pattern indicated above is developed with even parity. For example, an ASCII 'A' character is sent as:

| STOP | P | MSB | | | | | | LSB | START |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |

## EBCDIC

Extended binary coded data interchange code (EBCDIC), originally developed by IBM, uses 8 bits to represent each character. EBCDIC is similar in concept to the ASCII code, but specific bit patterns are different and it is incompatible with ASCII. When IBM introduced its personal computer range, they decided to adopt the ASCII Code, so EBCDIC does not have much relevance to data communications in the industrial environment. Refer to the EBCDIC Table 2.7.

## 4-bit binary code

For purely numerical data a 4-bit binary code, giving 16 characters ($2^4$), is sometimes used. The numbers 0–9 are represented by the binary codes 0000 to 1001 and the remaining codes are used for decimal points. This increases transmission speed or reduces the number of connections in simple systems. The 4-bit binary code is shown in Table 2.8.

| Decimal | Binary Code | Gray Code | BCD Code |
|---|---|---|---|
| 0 | 0000 | 0000 | 0000 |
| 1 | 0001 | 0001 | 0000 |
| 2 | 0010 | 0011 | 0000 |
| 3 | 0011 | 0010 | 0000 |
| 4 | 0100 | 0110 | 0000 |
| 5 | 0101 | 0111 | 0000 |
| 6 | 0110 | 0101 | 0000 |
| 7 | 0111 | 0100 | 0000 |
| 8 | 1000 | 1100 | 0000 |
| 9 | 1001 | 1101 | 0000 |
| 10 | 1010 | 1111 | 0001 0000 |
| 11 | 1011 | 1110 | 0001 0001 |
| 12 | 1100 | 1010 | 0001 0010 |
| 13 | 1101 | 1011 | 0001 0011 |
| 14 | 1110 | 1001 | 0001 0100 |
| 15 | 1111 | 1000 | 0001 0101 |

**Table 2.8**
*4-bit binary code*

## Gray code

Binary code is not ideal for some types of devices because multiple digits have to change every alternate count as the code increments. For incremental devices, such as shaft position encoders, which give a code output of shaft positions, the Gray code can be used. The advantage of this code over binary is that only one bit changes every time the value is incremented. This reduces the ambiguity in measuring consecutive angular positions. The Gray code is shown in Table 2.9.

| Decimal | Gray Code |
|---|---|
| 0 | 0000 |
| 1 | 0001 |
| 2 | 0011 |
| 3 | 0010 |
| 4 | 0110 |
| 5 | 0111 |
| 6 | 0101 |
| 7 | 0100 |
| 8 | 1100 |
| 9 | 1101 |
| 10 | 1111 |
| 11 | 1110 |
| 12 | 1010 |
| 13 | 1011 |
| 14 | 1001 |
| 15 | 1000 |

**Table 2.9**
*Gray code*

## Binary coded decimal

Binary coded decimal (BCD) is an extension of the 4-bit binary code. BCD encoding converts each separate digit of a decimal number into a 4-bit binary code. Consequently, the BCD uses 4 bits to represent one decimal digit. Although 4 bits in the binary code can represent 16 numbers (from 0 to 15) only the first 10 of these, from 0 to 9, are valid for BCD.

| Decimal | Binary Code | Gray Code | BCD Code |
|---|---|---|---|
| 0 | 0000 | 0000 | 0000 |
| 1 | 0001 | 0001 | 0001 |
| 2 | 0010 | 0011 | 0010 |
| 3 | 0011 | 0010 | 0011 |
| 4 | 0100 | 0110 | 0100 |
| 5 | 0101 | 0111 | 0101 |
| 6 | 0110 | 0101 | 0110 |
| 7 | 0111 | 0100 | 0111 |
| 8 | 1000 | 1100 | 1000 |
| 9 | 1001 | 1101 | 1001 |
| 10 | 1010 | 1111 | 0001 0000 |
| 11 | 1011 | 1110 | 0001 0001 |
| 12 | 1100 | 1010 | 0001 0010 |
| 13 | 1101 | 1011 | 0001 0011 |
| 14 | 1110 | 1001 | 0001 0100 |
| 15 | 1111 | 1000 | 0001 0101 |

**Table 2.10**
*Comparison of Binary, Gray and BCD codes*

BCD is commonly used on relatively simple systems such as small instruments, thumbwheels, and digital panel meters. Special interface cards and integrated circuits (ICs) are available for connecting BCD components to other intelligent devices. They can be connected directly to the inputs and outputs of PLCs.

A typical application for BCD is the setting of a parameter on a control panel from a group of thumbwheels. Each thumbwheel represents a decimal digit (from left to right; thousands, hundreds, tens and units digits). The interface connection of each digit to a PLC requires 4 wires plus a common, which would mean a total of 20 wires for a 4-digit set of thumbwheels. The number of wires, and their connections to a PLC, can be reduced to 8 by using a time division multiplexing system as shown in Figure 2.6. Each PLC output is energized in turn, and the binary code is measured by the PLC at four inputs. A similar arrangement is used in reverse for the digital display on a panel meter, using a group of four 7-segment LCD or LED displays.

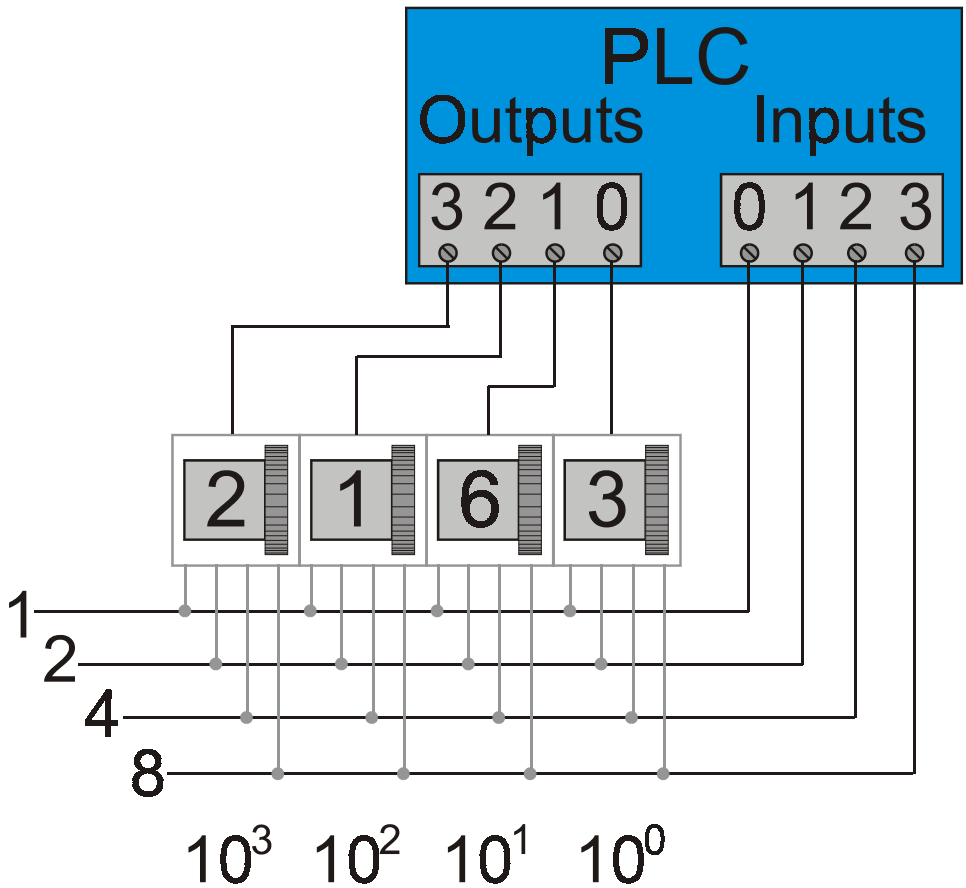



**Figure 2.6**
*BCD Thumbwheel switches and connections to PLC*

## 2.9 The universal asynchronous receiver/transmitter (UART)

The start, stop and parity bits used in asynchronous transmission systems are usually physically generated by a standard integrated circuit (IC) chip that is part of the interface circuitry between the microprocessor bus and the line driver (or receiver) of the communications link. This type of IC is called a **UART (universal asynchronous receiver/transmitter)** or sometimes an ACE (asynchronous communications element).

Various forms of UART are also used in synchronous data communications, called **USRT**. Collectively, these are all called **USART**s. The outputs of a UART are not designed to interface directly with the communications link. Additional devices, called line drivers and line receivers, are necessary to give out and receive the voltages appropriate to the communications link.

8250, 16450, 16550 are examples of UARTs, and 8251 is an example of a USART.

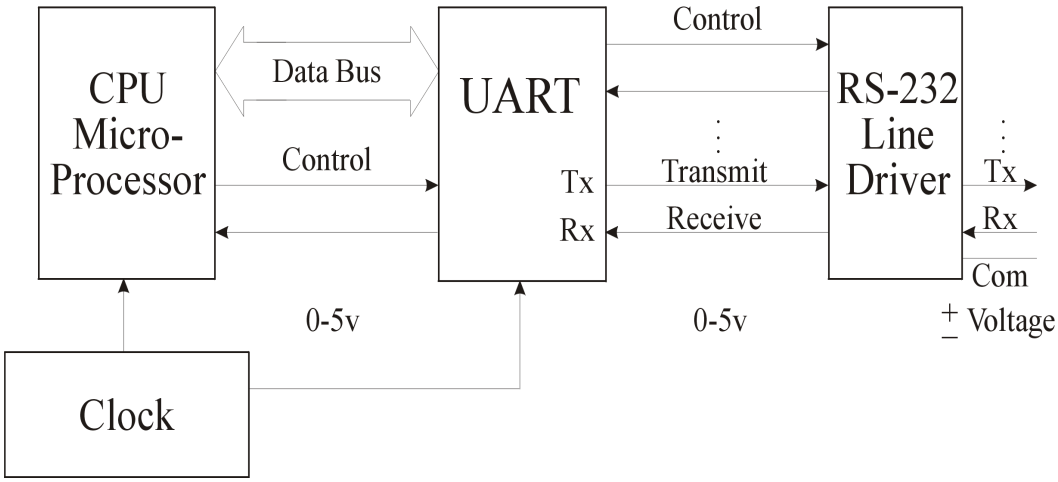



**Figure 2.7**
*Typical connection details of the UART*

The main purpose of the UART is to look after all the routine 'housekeeping' matters associated with preparing the 8 bit parallel output of a microprocessor for asynchronous serial data communication. The timing pulses are derived from the microprocessor master clock through external connections.

When **transmitting**, the UART:

- Sets the baud rate
- Accepts character bits from microprocessor as a parallel group
- Generates a start bit
- Adds the data bits in a serial group
- Determines the parity and adds a parity bit (if required)
- Ends transmission with a stop bit (sometimes 2 stop bits)
- Then signals the microprocessor that it is ready for the next character
- Coordinates handshaking when required

The UART has a separate signal line for transmit (TX) and one for receive (RX) so that it can operate in the full-duplex or a half-duplex mode. Other connections on the UART provide hardware signals for handshaking, the method of providing some form of 'interlocking' between two devices at the ends of a data communications link. Handshaking is discussed in more detail in Chapter 3.

When **receiving**, the UART:

- Sets the baud rate at the receiver
- Recognizes the start bit
- Reads the data bits in a serial group
- Reads the parity bit and checks the parity
- Recognizes the stop bit(s)
- Transfers the character as a parallel group to the microprocessor for further processing
- Coordinates handshaking when required
- Checks for data errors and flags the error bit in the status register

This removes the burden of programming the above routines in the microprocessor and, instead, they are handled transparently by the UART. All the program does with serial data is to simply write/read bytes to/from the UART.

## The UART transmitter

A byte received from the microprocessor for transmission is written to the I/O address of the UART's transmission sector. The bits to be transmitted are loaded into a shift register, then shifted out on the negative transition of the transmit data clock. This pulse rate sets the baud rate. When all the bits have been shifted out of the transmitter's shift register, the next packet is loaded and the process is repeated. The word 'packet' is used to indicate start, data, parity and stop bits all packaged together. Some authors refer to the packet as a serial data unit (SDU).

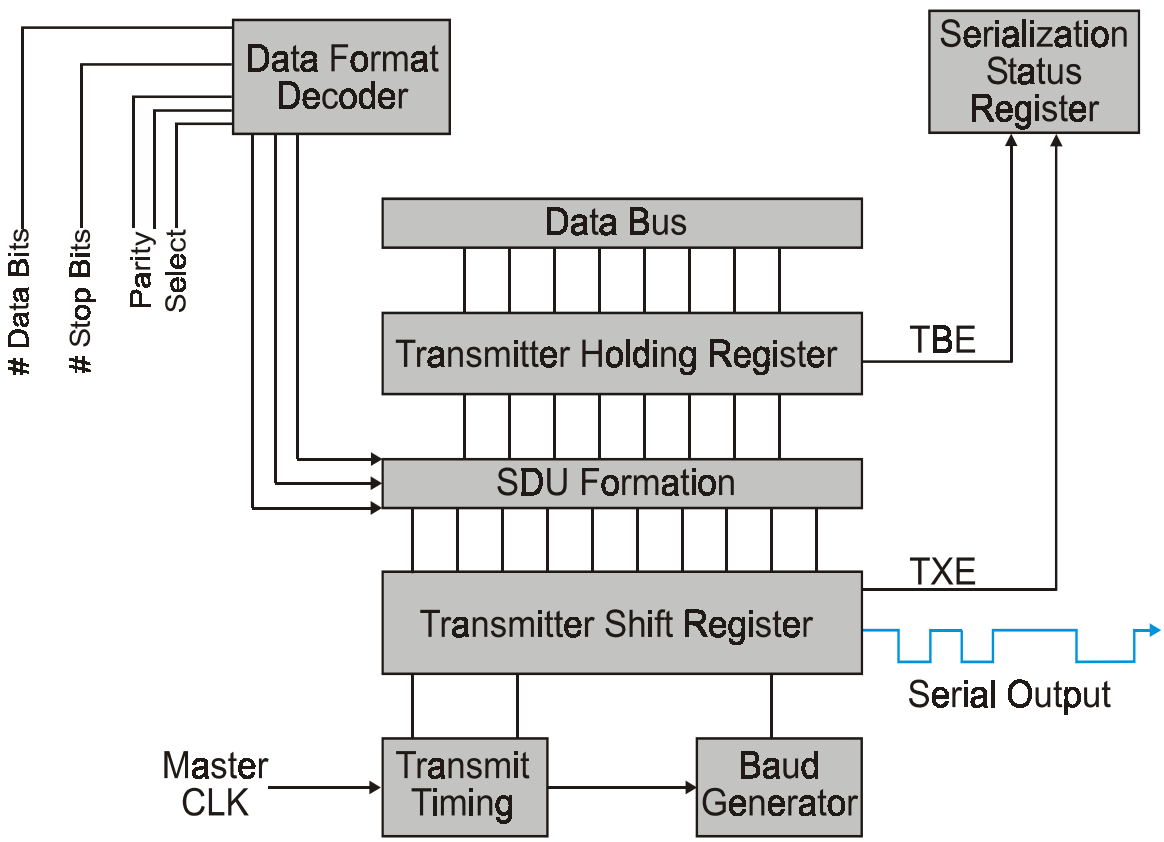



**Figure 2.8**
*The UART transmitter*

Between the transmitter holding register and the shift register is a section called the SDU (serial data unit) formation. This section constructs the actual packet to be loaded into the shift register.

In full duplex communications, the software needs to only test the value of the transmitter buffer empty (TBE) flag to decide whether to write a byte to the UART. In half-duplex communications, the modem must swap between transmitter and receiver states. Hence, the software must check both the transmitter buffer and the transmitter's shift register, as there may still be some data there.

## The UART receiver

The UART receiver continuously monitors the incoming serial line waiting for a start bit. When the start bit is received, the receiver line is monitored at the selected baud rate and the successive bits are placed in the receiver's shift register. This takes place according to the format described in the user programmable data format register. After assembly of the byte, it is moved into a FIFO (first in first out) buffer. At this stage, the RxRDY (receiver ready) flag is set **true** and remains true until all the contents of the FIFO buffer are empty.

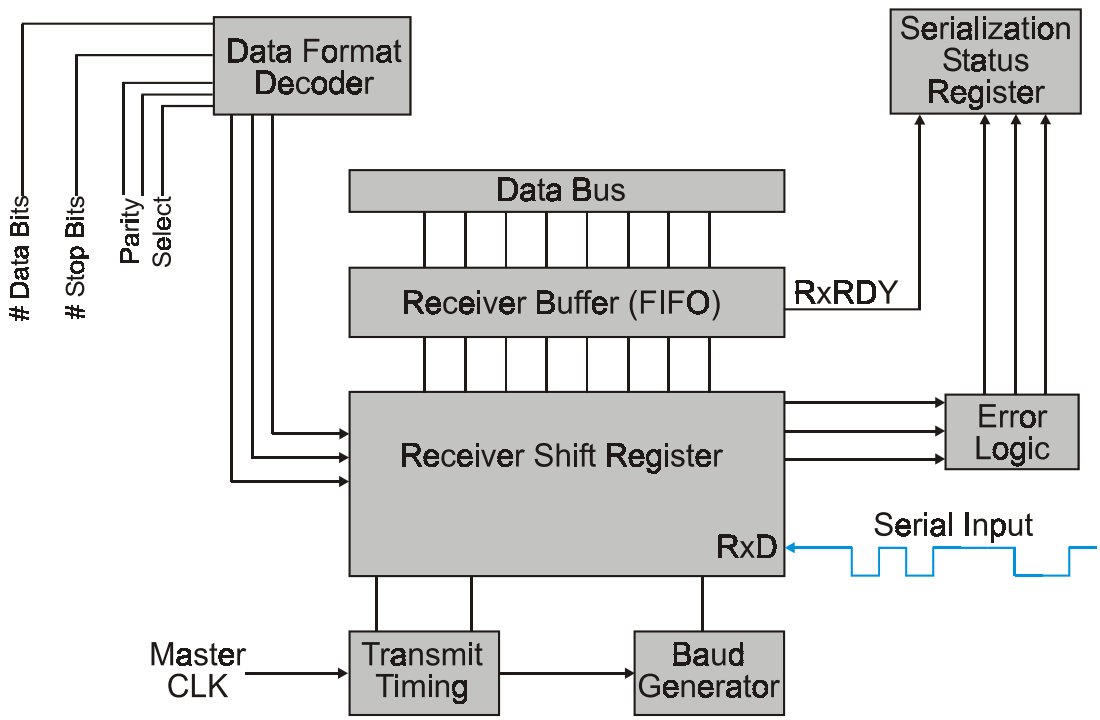



**Figure 2.9**
*The UART receiver*

## Typical errors

Another major function of the UART is to detect errors in the data received. Most errors are receiver errors. Typical errors are:

| | |
|---|---|
| Receiver overrun: | Bytes received faster than they can be read |
| Parity error: | Parity bit disagreement |
| Framing error: | This occurs if the detected bits do not fit into the frame selected |
| Break error: | This occurs if a start bit is detected for more than a frame time. |

## Break detect

To gain the attention of a receiver, a transmitter may hold the data line in a space condition (+voltage) for a period of time longer than that required for a complete character. This is called a break, and receivers can be equipped with a break detect to detect this condition. It is useful for interrupting the receiver, even in the middle of a stream of characters being sent. The break detect time is a function of the baud rate.

Serialization errors are reported in the serialization status register as shown in Figure 2.8 and Figure 2.9.

## Receiver timing

It is necessary to have separate clock signals for the UART's internal operations and to control the shifting operations in the transmitter and receiver sections. The frequency of the master signal is designed to be many times higher than that of the baud rate. This ratio of master serial clock to baud rate is called the clocking factor (typically 16). Instead of sampling the input line at the baud rate frequency, the improved start bit detector samples the incoming line at the rate of the master clock. This minimizes the possibility of an error due to slippage of sampling a stream of serial bits and sampling the wrong bit.

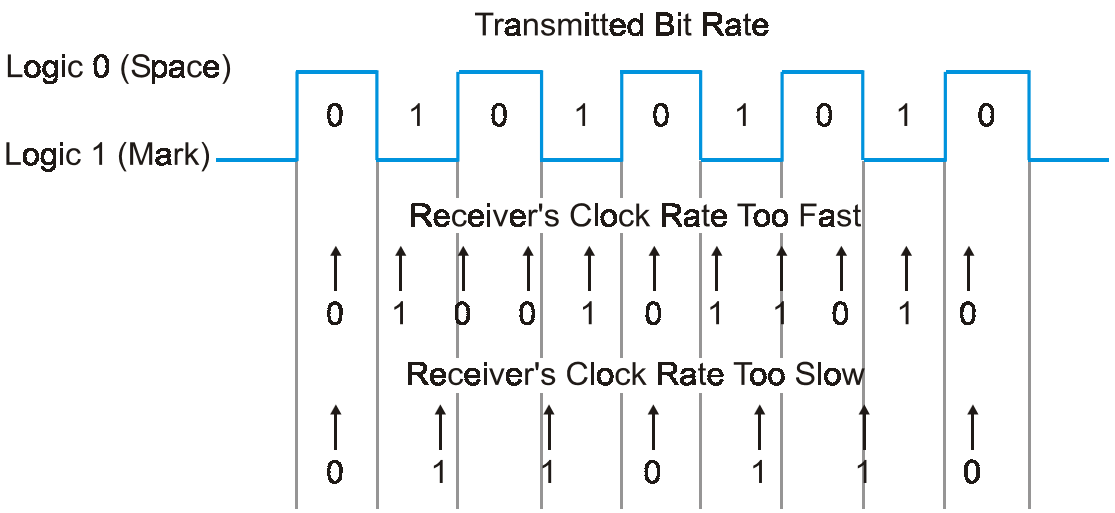



**Figure 2.10**
*Example of incorrect timing between source and receiver*

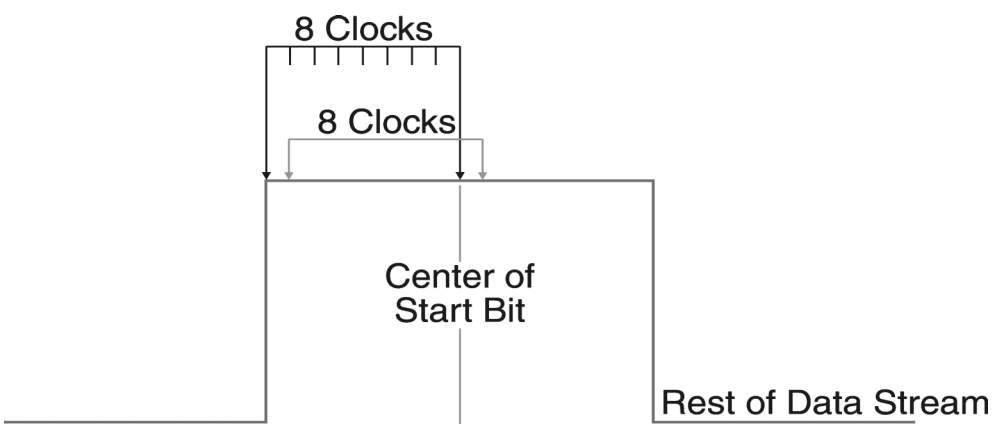



**Figure 2.11**
*Minimization of error with a clocking factor of 16*

The earliest serial ports used 8250 or 8251 chips, which interrupted the main processor for every character to be transmitted or received. This worked well for the speeds of that time. This has since been replaced by the 16450 chip which works in a similar fashion but supported faster PC bus speeds, and later by 16550 which has a 16-byte buffer thereby reducing the number of CPU interruptions by a factor of 16. A more recent development is to use an enhanced serial port, which provides a buffer of about 1000 bytes and has its own processor to reduce the interruptions to the main CPU by a factor of 1000.

## 2.10 The high speed UART (16550)

The 16550 is a high speed serial universal asynchronous receiver transmitter (UART). It is the default UART used on all IBM compatible computers and COM ports sold today. It varies greatly from the old 8250 UARTs in two ways: speed and the size of the FIFO buffer. The advantage of the 16550 over the older 16450 and 8250 UARTs is that it has a 16-byte buffer.

### Speed

The 16550 can operate at speeds from 1 to 115 k baud. The 16550 is commonly used on RS-232 even though the RS-232 standard only allows communication at speeds up to 19.2 k baud. Due to the availability and low cost of 16550 chips, the manufacturers of computers and add-on COM ports have included the 16550 as standard equipment.

### FIFO buffer

The old 8250 UART (19.2 k) had only a one-byte FIFO buffer. The advantages of the 16-byte buffer on the 16550 are twofold:

- The 16550 makes high speed communications much more reliable.
  On older chips, with their one-byte buffer, the UART would lose data if a second byte came in to the UART before the CPU had a chance to retrieve the first byte. The 16550, with its 16-byte buffer, gives the CPU up to 16 chances to retrieve the data before a character is lost. To realize what this means, if a UART is running at 19 200 bps (with a 10-bit character frame) the CPU will need to service the COM port 1920 times each second or once every .0005 seconds. If the CPU happens to take .0006 seconds to get around to servicing the COM port then the first byte is lost in the one-byte buffer UART. On the 16550 chip, with 16 bytes of buffer space, you can have up to 0.008 seconds to service the COM port.
- It helps make a multitasking system more efficient.
  When the COM port is transmitting data, it has to interrupt the CPU and fill the UART's transmitter buffer. That means that if the CPU is doing a background directory scan the scan will take longer while the COM port attempts to send data out to the outside world.

  In the one-byte buffer UARTs at 19 200 bps the COM port must interrupt the CPU 1920 times each second just to send data out of the COM port. With the 16550, however, it can put up to 16 bytes into the buffer at a time and therefore interrupt the CPU only 120 times each second. This increases the performance of the CPU to COM port system.